want to scale it dynamically. When a new version of the software is available, you'll need a reliable way of upgrading the service and migrating existing constructs and data. You could perform all those tasks manually. Or you could use an Operator to capture all those operational requirements and instruct Kubernetes to take care of them automatically. In practice, an Operator is an application that runs on Kubernetes and interacts with its API to accomplish its functionality.

The RabbitMQ project provides an official Operator to run the event broker on a Kubernetes cluster (www.rabbitmq.com). I have already configured all the necessary resources to use the RabbitMQ Kubernetes Operator and prepared a script to deploy it.

Open a Terminal window, go to your Polar Deployment project (polar-deployment), and navigate to the kubernetes/platform/production/rabbitmq folder. You should have copied that folder over to your repository when configuring the Kubernetes cluster. If that's not the case, please do so now from the source code repository accompanying this book (Chapter15/15-end/polar-deployment/platform/production/rabbitmq).

Then run the following command to deploy RabbitMQ to your production Kubernetes cluster:

```
$ ./deploy.sh
```

Feel free to open the file and look at the instructions before running it.

**NOTE** You might need to make the script executable first with the command `chmod +x deploy.sh`.

The script will output details about all the operations performed to deploy RabbitMQ. Finally, it will create a `polar-rabbitmq-`